Knowledge Graph Construction from *A Christmas Carol*

Melissa Maistro

July 2, 2026

1 Problem and approach

The goal is to build a small knowledge graph from unstructured text using a large language model with minimal human intervention. This mirrors the kind of task where structured knowledge is extracted from technical documents to support failure diagnosis. As a test document, I decided to use an excerpt from Charles Dickens' *A Christmas Carol* (Stave One).

My approach has three steps: clean the text, use a local LLM to extract (subject, relation, object) triples, and build a directed graph from those triples. The model has not been fine-tuned; it is used as-is and steered only through the prompt and a few-shot example, which fits the "minimal human intervention" and "prompt-based approach" requirements.

Model. I used Llama 3.1 8B (4-bit quantised) running locally through Ollama. Running locally keeps the data on the machine, which matters for sensitive documents. I set TEMPERATURE = 0 for deterministic output and forced valid JSON with Ollama's FORMAT="JSON" option, which removes most parsing errors. The full prompt was about 8,800 tokens, so I set the context window to 16,384 to make sure the whole excerpt fit without truncation.

Cleaning. A short script (CLEAN_EXCERPT.PY) removes inline page-number markers such as [12], collapses whitespace, and drops duplicate sentences that were paste artifacts. This is done programmatically so it is reproducible.

2 Prompt iterations

Most of the work was in the prompt. I ran four versions and compared them against a reference set of relationships I wrote by hand.

Prompt 1 (baseline). A simple instruction to extract triples. It returned 13 triples. Precision was high and it avoided the metaphor traps (for example it did not treat "dead as a door-nail" as a fact), but recall was low. It only captured the dense Scrooge and Marley cluster (PARTNER_OF, EXECUTOR_OF, FRIEND_OF, and so on) and missed almost every minor character. It also produced one broken triple, MARLEY – DEAD –> NULL, because it had a one-place fact and no object to put.

Prompt 2 (final version). I added three rules: always produce a non-empty object and express states as a relation (for example MARLEY – HAS_STATE –> DEAD); keep subjects and objects as single entities and split phrases into several triples; and extract every character including minor ones. I taught these with

a small worked example rather than only describing them. This raised the output to 23 triples and roughly doubled the recall. It now found the clerk, the nephew, the charity gentlemen, the chain, and the Three Spirits, and it split phrases correctly (for example Scrooge – lived_in –> chambers plus chambers – formerly_belonged_to –> Marley). The cost was a small drop in precision: one metaphor leaked (Marley's Ghost – travelled –> on the wings of the wind) and a few objects were still phrases.

Chunking experiment (rejected). To push recall further I split the text into overlapping, sentence-aligned chunks of about 800 to 1,500 tokens and ran the extraction on each chunk. This made the problem worse, not better. The output grew to about 190 triples but precision collapsed. With less surrounding context the model lost its sense of what mattered and started transcribing the plot, for example Scrooge – saw_poker –> poker, whole lines of dialogue as objects, and placeholder values like true and none. I kept this and the other rejected prompt versions in experiments/ as documented evidence. The lesson is that judging relevance depends on seeing the text as a whole, so smaller chunks trade coverage for a loss of selectivity.

Prompt 3 (selective, rejected). I then tried the opposite: a single pass with strict rules to exclude transient actions, dialogue, and scene description. This over-corrected. It dropped to 18 triples and lost real facts (the clerk disappeared, the nephew lost most of his relations), and it introduced new errors such as Scrooge – employed –> himself.

Conclusion. The four versions trace a precision and recall trade-off. Baseline was precise but narrow, prompt 2 was the best balance, chunking pushed recall too far into noise, and prompt 3 pushed selectivity too far into omission. I stopped at prompt 2 because the results were oscillating rather than improving, which suggested the 8B model had reached its limit on this text.

3 Manual cleanup

I did light manual cleanup on the 23 triples from prompt 2, going from 23 to 27 final triples. I fixed nothing that changed the meaning; I only corrected structural problems the model could not handle:

- Split the chain into a proper sub-graph: Marley's Ghost – was_bound_by –> chain, then one chain – made_of –> … edge per material (cash-boxes, keys, padlocks, ledgers, deeds, purses), instead of one long phrase object.
- Removed the leaked metaphor (on the wings of the wind) and one vague triple (gentlemen – represented –> Marley's liberality).

For this small set, hand-editing was faster and clearer than an automated pass. I kept the raw output as triples_raw.json so the before and after is visible.

4 Graph construction and visualization

build_graph.py reads the final triples and builds a directed multigraph with NetworkX, where each triple becomes a separate edge labelled by its relation. A multigraph is used deliberately so that the several relations between the same two entities (for example Scrooge is Marley's partner_of, executor_of, and friend_of) are all preserved rather than collapsed into one. The graph is exported as GraphML (a standard,

tool-independent format) and rendered as an interactive HTML page with pyvis (VISUALIZE_PYVIS.PY), where node size scales with degree and parallel edges are drawn as separate arcs.

The graph has 20 nodes and 27 edges. Scrooge is the main hub with 15 connections, followed by Marley with 9 and the chain with 7. This confirms numerically what the story shows: the text is centred on the Scrooge and Marley relationship, with smaller branches for the clerk, the nephew, the gentlemen, and the Ghost, plus a small sub-tree for the chain and its materials.

5 Evaluation

Against my reference set, the final graph has high precision (nearly every triple is a fact stated in the text) and good recall across the main cast. It captures all the durable relationships that define the characters. It does not capture a few single-mention details such as the firm name "Scrooge and Marley" as an entity, but overall the graph is small, clean, and faithful to the source.

6 Challenges and possible improvements

- *Precision versus recall.* Finding minor characters without also extracting noise was the central difficulty, as the four prompt versions show. A schema constraining the output (a fixed relation vocabulary or structured output tools) would make this more stable.
- *Coreference.* Minor characters are unnamed ("the clerk", "the nephew") and referred to by pronouns, which the model resolves inconsistently. A dedicated coreference step before extraction would help.
- *Structure mismatch.* Some facts do not fit the rigid triple shape, such as states (handled with HAS_STATE) and multi-part facts (the chain), needing prompt rules or manual splitting. Automated entity resolution (fuzzy matching or embeddings) would replace the manual cleanup at scale.
- *Metaphor.* The model sometimes extracts figures of speech as facts even when told not to. A larger or newer model, or a two-stage approach (entities first, then relations), would likely reduce this.
- *Performance and scale.* On a CPU-only laptop one extraction took several minutes. For long real-world documents, chunking combined with a stronger relevance filter would improve recall without the plot-transcription problem seen here, and with a labeled domain corpus, fine-tuning a small model on the specific relation types would be worth evaluating over prompting alone.